

Introduction to the Linux Course

23/11/2025

Topics

- **Linux history and Linux distros**

Link: <https://www.youtube.com/watch?v=ShcR4Zfc6Dw>

- **Unix philosophy**

Link: <https://youtu.be/gojeTqXdBH0?si=kg-iP4KEFjLU-Tto>

- **Kernel vs OS**

Link: <https://youtu.be/IvGdY6luTtU?si=nEcir9iqjmYMa1lo>

- **Free Software Philosophy**

Link: <https://youtu.be/Ag1AKIl2GM?si=0YDnS2BDSrAAMFX8>

- **Why programmers love Linux**

Link: https://youtu.be/otDOHt_Jges?si=4ChY6-xpYbOdbKQ6

- **Pros and cons**

Link: https://youtu.be/pYfzZRyRzvs?si=4Q5_H0gJ3JHBod0c

Introduction to Linux

Welcome to the first step of your Linux journey. Whether you're a developer, cybersecurity student, or someone who just wants to understand how computers *really* work, this course will give you the foundation you need to actually **use** Linux—not just read about it.

This introduction will help you understand:

- What Linux actually is
- Why it became such a big deal

- How it's different from other operating systems
- Why developers, hackers, and engineers prefer it
- What mindset you should adopt while learning Linux
- How we'll approach the workshop

If this is your first time touching Linux: **perfect**. This course is made for you.

1. Why Linux Matters Today

If you look at the modern world—servers, cloud platforms, supercomputers, phones, routers, IoT, cybersecurity labs, programming environments—Linux is *everywhere*. It's the invisible engine running most of the technologies you interact with daily.

A few facts:

- **100%** of the world's top 500 supercomputers run Linux.
- **Most web servers** use Linux (Apache, Nginx).
- **Android**, the most widely-used mobile OS, is built on the Linux kernel.
- **Cybersecurity distributions** (Kali, Parrot) are Linux-based.
- Developers choose Linux for transparency, speed, customizability, and control.

So even if you've never installed Linux on your laptop, you've definitely used it indirectly.

2. Understanding Linux: What It Really Is

One of the biggest misconceptions is thinking Linux is "that thing that replaces Windows." Linux is not a program. It's not a GUI. It's not even an operating system by itself.

Linux = the kernel

The kernel is the core program that:

- Talks to hardware
- Manages memory

- Schedules processes
- Handles permissions
- Controls I/O

Everything else—the desktop, apps, package manager, and utilities—comes from the broader free-software ecosystem called **GNU**.

Together: **GNU tools + Linux kernel = GNU/Linux (a complete OS)**.

This separation is important because Linux teaches you something deeper than using an OS—it teaches you *how systems work*.

3. Quick Overview of the UNIX Philosophy

Linux inherits its design principles from Unix. This philosophy will shape how you think during the whole workshop:

The UNIX Philosophy

1. **Write programs that do one thing, and do it well.**
2. **Write programs to work together.**
3. **Write programs to handle text, because text is a universal interface.**

This leads to:

- Tools that are small and powerful
- Commands that do simple tasks
- The magic happens when you combine them using pipes (|)

It's why a Linux power-user is unstoppable—every small tool becomes a Lego block, and you can build anything from those blocks.

4. Linux History (The Simple Version)

Let's keep it practical.

1969 – Unix born

ATT created Unix. Clean, elegant, powerful.

1983 – GNU Project launched

Richard Stallman began building a free Unix-like system. GNU had everything—compilers, libraries, tools—*except a kernel*.

1991 – Linus Torvalds writes the Linux Kernel

As a personal hobby project. He released it under a free license, allowing anyone to study, modify, and contribute.

1992 – GNU + Linux merged

This created the first fully free operating system.

1990s–2000s – Linux takes over servers

Why? Stability, control, security, no licensing fees.

2010s–2020s – Linux becomes everywhere

Cloud, containers, mobile, embedded systems, DevOps... The world quietly standardized on Linux.

That's why learning Linux today is a **career skill**, not just a personal preference.

5. Linux Distributions (Distros)

If Linux is the engine, a **distro** is the full car.

Different distros = different combinations of:

- Package managers
- Desktop environments
- Default tools
- Philosophy
- Target audience

Beginner-friendly distros

- Ubuntu
- Linux Mint
- MX Linux

Power-user distros

- Arch Linux
- Gentoo
- Void Linux

Enterprise distros

- Red Hat Enterprise Linux
- SUSE Linux Enterprise

Security distros

- Kali Linux
- Parrot OS

Throughout the workshop, we'll use a distro that is: Stable, Easy to install, Common across many environments (Usually Ubuntu or Mint—you'll get confirmation before the workshop.)

6. Kernel vs Operating System

This is where newcomers usually get confused.

Kernel

- The lowest-level component
- Manages memory, processes, scheduling
- Talks to CPU, RAM, disks
- Loads drivers
- Handles system calls

Operating System

- Kernel
- Plus userland tools (GNU utilities)
- Plus services
- Plus package managers
- Plus desktop environments
- Plus apps

Linux is the *kernel*. GNU/Linux is the *operating system*.

This distinction matters because in this course you will interact with the kernel indirectly using tools like:

- `top`, `ps`, `dmesg`
- `/proc` filesystem
- System calls through C programs
- Permissions and user management

This duality is key to becoming a real system administrator or developer.

7. The Free Software Philosophy (Why Linux Exists)

Linux exists because of the free software movement. Free software is about *freedom*, not price.

The Four Essential Freedoms

1. **Freedom 0:** Run the program for any purpose
2. **Freedom 1:** Study the source code and modify it
3. **Freedom 2:** Distribute copies
4. **Freedom 3:** Distribute modified versions

If a program denies any of these freedoms, it becomes **proprietary**, which means:

- You can't see how it works

- You can't fix it
- You depend on the vendor
- You lose control over your own computer

Linux was built to solve this.

During the workshop, you'll see the impact of this philosophy:

- Software you can fully audit
- Tools you can modify
- Community-built contributions
- Transparency at every layer

This is why cybersecurity professionals prefer Linux—you can see *everything* happening in the system.

8. Why Programmers Love Linux

Software developers love Linux for reasons that will become obvious once you start using it:

1. Everything is a file

Devices, sockets, processes, IPC... all represented as files. Simplifies development massively.

2. Powerful terminal environment

You can automate anything.

3. Package managers

Install complete development stacks in seconds.

4. DevOps and cloud-native tools

Docker, Kubernetes, CI pipelines—all built on Linux.

5. Open-source ecosystem

Learn from real-world code. Fix things. Build tools.

6. High performance and stability

Perfect for servers and embedded systems.

7. Customizability

Modify your environment to match exactly how you think and work.

This course will help you feel comfortable enough that Linux becomes your main development environment.

9. Pros and Cons of Linux

Pros

- Powerful CLI
- Free and open source
- Fast, lightweight
- Secure and stable
- Full control over your system
- Great for programming
- Great for networking
- Massive community support

Cons

- Some hardware drivers might be missing
- Gaming is improving but still imperfect
- Adobe apps not officially available
- Certain distros can be difficult to use
- Requires learning mindset, not point-and-click

But as a developer, engineer, or cybersecurity student, the *pros* outweigh the *cons* by a huge margin.

10. How to Think Like a Linux User

To succeed in this workshop:

Adopt these habits:

- **Be curious** — try commands, explore folders.
- **Read error messages** — Linux errors are usually helpful.
- **Use man pages** — every command has documentation.
- **Experiment safely** — break things in virtual machines.
- **Think in small tools** — pipes, filters, text processing.
- **Love the terminal** — it's your real power.

Linux rewards people who explore.

11. What You'll Learn in This Course

This introduction sets the ground for the entire course. By the end of the workshop, you will:

System Skills

- Install and manage Linux
- Understand filesystems
- Work confidently in the terminal
- Manage users, processes, permissions
- Control systemd services
- Understand the kernel's role

Networking Skills

- Configure network interfaces
- Use diagnostic tools (ping, traceroute, curl)
- Manage firewall rules

Programming and Automation

- Shell scripting
- Using package managers
- Working with logs
- Troubleshooting

Security Awareness

- Permissions
- Privilege escalation basics
- Monitoring system behavior

You'll not just use Linux—you'll **understand** Linux.

Useful Resources

YouTube Channels

- anaHr — <https://www.youtube.com/@anaHr>
- mohammad besar — <https://www.youtube.com/@mmbesar>
- alwaqqad — <https://www.youtube.com/@alwaqqad>
- sudostart — https://www.youtube.com/@Sudo_Start
- linuxtopia — <https://www.youtube.com/@linuxtopia>
- abdulmogeeb alhameed — <https://www.youtube.com/@abdulmogeeb>

Books

- **The Art Of UNIX Programming** — *Eric Raymond*